Пульс Проекта

ПЛАНЫ И СРОКИ ПРОЕКТОВ

Руководство по развёртыванию

Версия 0.7.3

Октябрь 2026

© Alexander Yurasov

Содержание

1 О документе 3

2 Архитектура 4

3 Требования 5

3.1 Сервер 5

3.2 Клиенты 5

4 Установка 6

4.1 Шаг 1. Получение дистрибутива 6

4.2 Шаг 2. Конфигурация 6

4.3 Шаг 3. Сборка и запуск 6

4.4 Шаг 4. Проверка 6

4.5 Шаг 5. HTTPS 6

4.6 Шаг 6. Первый вход 7

5 Переход с 1.0.1 8

6 Переменные окружения 9

6.1 Основные 9

6.2 Сессии и безопасность 9

6.3 Работа сервиса 10

6.4 Интеграции: Jira и «По Делу» 10

6.5 Резервное копирование 11

7 Структура компании из командной строки 12

8 Резервное копирование и восстановление 13

8.1 Копии 13

8.2 Восстановление 13

9 Обновление 15

9.1 Обычный порядок 15

9.2 Откат 15

10 Наблюдение и журналы 16

10.1 Проверки 16

10.2 Журналы 16

11 Безопасность 17

12 Масштабирование 18

13 Рядом с другими системами на одной машине 19

14 Установка в закрытом контуре 22

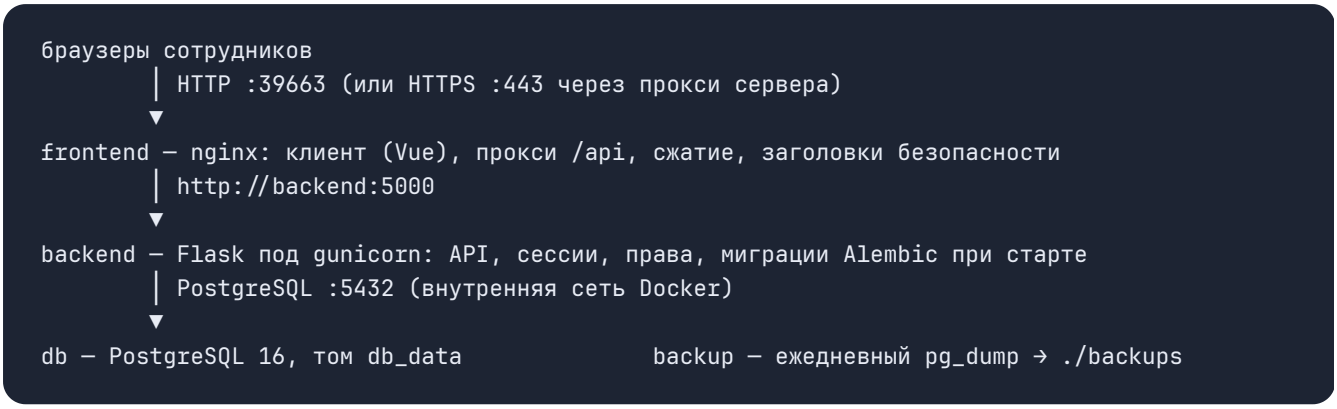
15 Диагностика 23

16 Справочник команд 25

1 О документе

Руководство описывает установку «Пульса Проекта» версии 0.4.0 на сервер компании, переход с версии 1.0.1, настройку HTTPS, резервное копирование и восстановление, обновление, наблюдение и диагностику. Документ предназначен для инженеров сопровождения. Работа в системе описана в руководствах пользователя и администратора.

2 Архитектура



Наружу открыт только порт frontend. Backend и база доступны лишь внутри сети Docker Compose. Сервис `legacy-import` (профиль `migrate`) запускается вручную один раз — для переноса базы 1.0.x.

3 Требования

3.1 Сервер

ПАРАМЕТР	ЗНАЧЕНИЕ
ОС	Linux x86-64 с Docker Engine (проверено на Ubuntu 24.04; подойдёт любой дистрибутив с Docker Compose V2)
Docker	Docker Engine 24+ с Compose V2 (`docker compose`)
Процессор и память	2 vCPU и 2 ГБ ОЗУ — до нескольких сотен сотрудников; 4 vCPU и 4 ГБ — 1000–2000 сотрудников
Диск	5 ГБ под образы и базу плюс место под резервные копии (≈ 4 МБ на копию на данных 2026 г.)
Сеть	Порт 39663 (или 443 через HTTPS-прокси); при сборке — доступ к реестру образов, PyPI и npm (или закрытый контур, см. ниже); для связи с Jira — доступ backend к адресу Jira (напрямую или через прокси `HTTPS_PROXY`)

Нагрузка измерена: на 4 vCPU — около 500 опросов изменений в секунду при задержке p95 < 100 мс; для 2000 сотрудников в пиковый час нужно около 110 запросов в секунду (`docs/performance/loadtest-v0.2.0.md`).

3.2 Клиенты

Актуальные Chrome, Яндекс Браузер, Edge, Firefox, Safari. Экран — от 1280 px по ширине.

4 Установка

4.1 Шаг 1. Получение дистрибутива

```
sudo git clone --branch v0.7.3 <адрес репозитория puls_proekta> /opt/puls
cd /opt/puls
```

4.2 Шаг 2. Конфигурация

```
sudo cp .env.example .env
sudo chmod 600 .env
openssl rand -hex 32   # → SECRET_KEY
openssl rand -hex 24   # → POSTGRES_PASSWORD
sudo nano .env
```

Обязательно: `SECRET_KEY` (не короче 32 символов), `POSTGRES_PASSWORD`. Для новой установки без переноса данных — `BOOTSTRAP_ADMIN_PASSWORD` (пароль первого администратора с логином `admin`; после первого входа смените его в профиле и очистите переменную). Остальное — раздел «Переменные окружения».

4.3 Шаг 3. Сборка и запуск

```
sudo docker compose up -d --build
sudo docker compose ps
```

Через 30–60 секунд `db`, `backend`, `frontend` — в состоянии `healthy`, `backup` — `running`. Backend при старте применяет миграции базы.

4.4 Шаг 4. Проверка

```
curl -fsS http://127.0.0.1:39663/api/health/ready; echo
# {"status":"ok","schema":"0008_user_capabilities","version":"0.7.3"}
curl -sI http://127.0.0.1:39663/ | grep -i content-security-policy
```

Откройте `http://<сервер>:39663` и войдите.

4.5 Шаг 5. HTTPS

Система не должна быть доступна по открытому HTTP за пределами внутренней сети. Варианты:

Caddy на сервере — `Caddyfile.example` в корне репозитория:

```
puls.company.ru {
    encode gzip
    header Strict-Transport-Security "max-age=31536000"
    reverse_proxy 127.0.0.1:39663
}
```

nginx на сервере (как у версии 1.0.1):

```
server {
    listen 443 ssl;
    server_name puls.company.ru;
    ssl_certificate     /etc/ssl/puls.crt;
    ssl_certificate_key /etc/ssl/puls.key;
    client_max_body_size 25m;
    location / {
        proxy_pass http://127.0.0.1:39663;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

После включения HTTPS в `.env`: `HTTP_BIND=127.0.0.1` (порт 39663 не виден снаружи), `SESSION_COOKIE_SECURE=true`, `TRUSTED_PROXY_HOPS=2` (адрес сотрудника — из заголовка прокси; от него зависит ограничение попыток входа), затем `sudo docker compose up -d`.

4.6 Шаг 6. Первый вход

- **Переход с 1.0.1** — администраторы и сотрудники входят прежними логинами и паролями (раздел «Переход с 1.0.1»).
- **Новая установка** — логин `admin`, пароль из `BOOTSTRAP_ADMIN_PASSWORD`. Смените пароль, заведите направления, сотрудников и доступы (руководство администратора).

5 Переход с 1.0.1

Подробная процедура с репетицией, проверками и откатом — `docs/runbooks/v0.2.0-migration-from-1.0.1.md`. Кратко:

1. Заранее: установка в `/opt/puls` (шаги 1–3 без запуска frontend: `docker compose build && docker compose up -d db`), репетиция переноса на копии рабочей базы с `--dry-run`.
2. Остановка 1.0.1: `sudo systemctl stop puls && sudo systemctl disable puls`; архив `/puls_proekta/puls.db*`.
3. Перенос: `sudo cp -a /puls_proekta/puls.db* /opt/puls/import/` → `sudo docker compose --profile migrate run --rm legacy-import` → отчёт `import/import-report.md`.
4. Запуск: `sudo docker compose up -d`, проверка, резервная копия `after-migration`.
5. Откат: `sudo docker compose down` (без `-v`) и `sudo systemctl enable --now puls`.

Перенос работает только в пустую базу и одной транзакцией: при любом расхождении сверки ничего не записывается (код завершения 2).

6 Переменные окружения

6.1 Основные

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`POSTGRES_USER`	`puls`	Пользователь PostgreSQL
`POSTGRES_DB`	`puls`	База PostgreSQL
`POSTGRES_PASSWORD`	—	Пароль PostgreSQL (обязателен)
`SECRET_KEY`	—	Секрет подписи, не короче 32 символов (обязателен)
`HTTP_PORT`	`39663`	Порт системы на сервере
`HTTP_BIND`	`0.0.0.0`	Адрес; `127.0.0.1` — только для прокси на этом же сервере
`TZ`	`Europe/Moscow`	Часовой пояс контейнеров
`BOOTSTRAP_ADMIN_PASSWORD`	—	Только новая установка: пароль администратора `admin` для пустой базы

6.2 Сессии и безопасность

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`SESSION_COOKIE_SECURE`	`false`	`true` — cookie только по HTTPS
`SESSION_LIFETIME_DAYS`	`30`	Срок жизни сессии
`LOGIN_RATE_LIMIT`	`10 per minute;50 per hour`	Попыток входа на одну учётную запись (на каждый процесс backend)
`LOGIN_IP_RATE_LIMIT`	`300 per minute`	Попыток входа с одного адреса — с запасом на офис за NAT
`TRUSTED_PROXY_HOPS`	`1`	Прокси перед backend: 1 — nginx контейнера; 2 — плюс HTTPS-прокси на сервере

6.3 Работа сервиса

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`SNAPSHOT_INTERVAL_MINUTES`	`30`	Автоснимок плана при правках не чаще раза в N минут
`GUNICORN_WORKERS`	2 × ядра + 1, не больше 9	Процессов backend
`AUTO_MIGRATE`	`true`	Применять миграции при старте backend
`ACCESS_LOG`	`false`	Журнал запросов backend

6.4 Интеграции: Jira и «По Делу»

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`JIRA_ALLOWED_HOSTS`	—	Разрешённые адреса Jira через запятую: `jira.company.ru` или `.company.ru` (домен и поддомены); пусто — любой
`JIRA_CACHE_SECONDS`	`300`	Сколько секунд данные задач Jira считаются свежими при открытии плана
`PODELU_ALLOWED_HOSTS`	—	Разрешённые адреса «По Делу» — так же, как `JIRA_ALLOWED_HOSTS`
`PODELU_TOTAL_TIMEOUT`	`50`	Сколько секунд одна загрузка из «По Делу» в интерфейсе может ждать её ответа целиком (вход, выгрузки, выход); меньше `GUNICORN_TIMEOUT` (60). Командная строка `sync-podelu` ждёт до 600 секунд
`HTTPS_PROXY`, `HTTP_PROXY`, `NO_PROXY`	—	Прокси для выхода backend к Jira и «По Делу», если сервер ходит в сеть через прокси. «По Делу» на той же машине — в `NO_PROXY`

Сами подключения (адрес, учётная запись, пароль, токен или ключ) задаёт администратор в интерфейсе: блок **Интеграции** бокового меню. Пароли и ключи других систем хранятся в базе зашифрованными ключом из `SECRET_KEY`: после смены `SECRET_KEY` их нужно ввести заново.

Ссылки из задач «По Делу» (0.5.0, нужна «По Делу» 1.4.1 или новее) работают в обе стороны: backend «Пульса» обращается к «По Делу» по `/api/integrations/puls/…`, а backend «По Делу» — к «Пульсу» по `/api/integration/v1/…` с заголовком `Authorization: Bearer puls_…` и без cookie. Обратный прокси перед «Пульсом» должен пропускать `/api/integration/` так же, как остальной `/api/` (в `nginx.conf` и `Caddyfile.example` из поставки так и есть), а сеть —

разрешать запросы между серверами в обе стороны. Адреса «По Делу» ограничивает `PODELU_ALLOWED_HOSTS`, адреса «Пульса» на стороне «По Делу» — её переменная `PULS_ALLOWED_HOSTS`. Проверка: «Интеграции → «По Делу» → Проверить связь» в «Пульсе» и «Проверить подключение» в «По Делу».

Запросы идут от сервера к серверу, а не из браузера: адрес, который открывают сотрудники, серверу может быть не виден. Если «По Делу» работает на той же машине, укажите в обеих системах **Адрес для запросов сервера** — раздел «Рядом с другими системами на одной машине», пункт «Связь «Пульса» и «По Делу»».

С 0.6.0 к `/api/integration/v1/…` обращаются и другие системы — например, «Дашборды компании» для сводного отчёта по проектам: каждая со своим ключом из «Интеграции → Ключи доступа» и своими правами. Ограничение частоты — 120 запросов в минуту на ключ, запросы без действующего ключа ограничиваются по адресу клиента. Миграция `0006_integration_keys` при обновлении переносит ключ «По Делу» в таблицу ключей — вводить его в «По Делу» заново не нужно.

6.5 Резервное копирование

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`BACKUP_DIR`	`./backups`	Каталог копий на сервере
`BACKUP_HOUR`	`03`	Час ежедневной копии (по `TZ`) — пока администратор не задал расписание в «Пульсе»
`BACKUP_RETENTION_DAYS`	`30`	Сколько дней хранить ежедневные копии — пока не задано в «Пульсе»

С 0.7.3 расписание (время и дни недели) и сроки хранения администратор задаёт в разделе **Администрирование → Резервные копии**; `BACKUP_HOUR` и `BACKUP_RETENTION_DAYS` действуют, пока он их там не сохранил.

7 Структура компании из командной строки

Те же операции, что на странице «Импорт и экспорт», — командой в контейнере backend (без `--apply` — только проверка, ничего не записывается):

```
docker compose exec backend python scripts/org_transfer.py export-structure -o
/tmp/structure.json
docker compose exec backend python scripts/org_transfer.py export-employees --format zip -o
/tmp/employees.zip
docker compose cp backend:/tmp/structure.json backend:/tmp/employees.zip .
```

Загрузка: скопируйте файл в контейнер и проверьте, затем примените (`--actor` — логин администратора, от имени которого изменения попадут в реестр):

```
docker compose cp people.csv backend:/tmp/people.csv
docker compose exec backend python scripts/org_transfer.py import-csv /tmp/people.csv --
create-missing --actor admin
docker compose exec backend python scripts/org_transfer.py import-csv /tmp/people.csv --
create-missing --actor admin --apply
```

Так же работают `import-structure` (`--no-settings` — без типов и глубины из файла), `import-employees` (`--new-users active|inactive|file`) и `sync-ad` (`--create-missing` , `--deactivate-missing`).

Синхронизация с Active Directory по расписанию — запись cron на сервере, например ежедневно в 06:15:

```
15 6 * * * cd /opt/puls && docker compose exec -T backend python scripts/org_transfer.py
sync-ad --create-missing --actor admin --apply >> /var/log/puls-ad-sync.log 2>&1
```

Подключение к каталогу берётся из «Интеграции → Вход через домен», параметры синхронизации — со страницы «Импорт и экспорт → Active Directory».

8 Резервное копирование и восстановление

8.1 Копии

Контейнер `backup` — служба резервного копирования. Раз в минуту она сверяется с расписанием из «Пульса» (**Администрирование → Резервные копии**: время, дни недели, сколько хранить копии) и делает полную копию базы (`pg_dump -Fc`) в `BACKUP_DIR` . Пока расписание в «Пульсе» не сохранено — ежедневно в `BACKUP_HOUR` :00, хранение `BACKUP_RETENTION_DAYS` дней. Если служба была остановлена во время копии по расписанию, она сделает её сразу после запуска в тот же день. Файлы — `puls_<дата-время>_<метка>.dump` (метки `daily` — по расписанию, `manual` — вручную и по кнопке «Сделать копию сейчас», `pre-update` , `pre-restore`), права 600 (в копии — хэши паролей и переписка). Старые копии удаляются после очередной копии: по расписанию — через заданный срок, остальные — через свой срок (0 — не удаляются).

```
cd /opt/puls
sudo docker compose exec backup sh /usr/local/bin/backup.sh now manual   # копия сейчас
sudo docker compose exec backup sh /usr/local/bin/backup.sh list          # список
```

`deploy/update.sh` делает копию `pre-update` перед каждым обновлением, восстановление — копию `pre-restore` .

Служба и «Пульс» обмениваются через базу (`app_settings`), приложению не нужны ни файлы копий, ни доступ к Docker: расписание — `backup_settings` , запрос «Сделать копию сейчас» — `backup_request` (служба выполняет его в течение минуты), а служба пишет в `backup_status` итог каждой копии, список копий, действующее расписание и раз в 5 минут — отметку «служба работает». Это показывают разделы «Резервные копии» и «Состояние системы»: служба не отмечалась больше 15 минут, последняя копия не удалась или копии нет дольше, чем по расписанию, — красная карточка. Если записать итог не удалось (база недоступна), копия всё равно остаётся, а в журнале контейнера — строка «не удалось записать состояние в базу».

> **Важно.** Копии на том же сервере не спасут от его потери. Копируйте `BACKUP_DIR` на другой сервер, например ежедневно cron-задачей: `rsync -a --chmod=F600 /opt/puls/backups/ backup@storage:/backups/puls/` .

8.2 Восстановление

```
cd /opt/puls
sudo docker compose stop backend frontend
sudo docker compose exec backup sh /usr/local/bin/backup.sh restore puls_20261030-
030000_daily.dump
sudo docker compose start backend frontend
```

Скрипт проверяет файл, отказывается работать, пока к базе подключён backend, сохраняет текущее состояние (`pre-restore`) и восстанавливает базу одной транзакцией. Восстановление проверено на копии production-данных.

> **Совет.** Раз в квартал восстанавливайте последнюю копию на тестовом сервере — это единственная проверка, что копии пригодны.

9 Обновление

9.1 Обычный порядок

1. Прочитайте `docs/versions_admin.md` и release notes новой версии.
2. Выполните:

```
cd /opt/puls
sudo sh deploy/update.sh v0.7.3
```

Скрипт: копия `pre-update` → получение версии из git (ветка — только перемоткой вперёд) → сборка и перезапуск (`docker compose up -d --build`, миграции при старте backend) → ожидание готовности до 5 минут.

3. Проверьте: `curl http://127.0.0.1:39663/api/health/ready` (версия), вход, открытие крупного проекта.

9.2 Откат

Если обновление не удалось, скрипт печатает команды отката:

```
git checkout <прежний коммит> && sudo docker compose up -d --build
# если миграции успели изменить базу:
sudo docker compose stop backend
sudo docker compose exec backup sh /usr/local/bin/backup.sh restore <файл pre-update>
sudo docker compose start backend
```

`.env` при обновлении не меняется; новые переменные описаны в `docs/versions_admin.md`.

Откат с 0.7.x на 0.6.x — восстановлением копии `pre-update`, как выше: правки, сделанные после обновления, при этом пропадут. Откат схемы без копии сохраняет правки — пока работает ещё 0.7.x, до перехода на прежний код:

```
sudo docker compose exec backend alembic downgrade 0006_integration_keys
```

Но у каждой строки остаётся только первая связь, связи с последующими вехами теряются, лаги в рабочих днях становятся календарными, а версии плана, сохранённые в 0.7.x, восстанавливаются в 0.6.x без связей.

10 Наблюдение и журналы

10.1 Проверки

АДРЕС	ЧТО ПРОВЕРЯЕТ
`GET /healthz`	nginx отвечает
`GET /api/health/live`	backend запущен
`GET /api/health/ready`	backend и база доступны, схема и версия

Контейнеры проверяют себя сами (`docker compose ps` — `healthy`) и перезапускаются после сбоя (`restart: unless-stopped`).

Сводку для людей — схема базы, резервные копии, ошибки сервера за сутки, подключения к Jira, «По Делу» и каталогу, производственный календарь — показывает раздел **Администрирование → Состояние системы** (администратору и сотрудникам с полномочием «Смотрит состояние системы»). Его данные — `GET /api/system/health` под сеансом такого сотрудника.

10.2 Журналы

```
sudo docker compose logs -f backend          # API, ошибки, миграции
sudo docker compose logs --tail 200 frontend # nginx
sudo docker compose logs backup              # резервные копии
```

Журнал запросов backend выключен; включить — `ACCESS_LOG=true` в `.env` и `sudo docker compose up -d` .

11 Безопасность

- `.env` — права 600, не копируйте в git и почту; `SECRET_KEY` уникален для каждой установки.
- Включите HTTPS и `SESSION_COOKIE_SECURE=true`.
- Наружу — только порт frontend; PostgreSQL наружу не публикуется.
- Backend работает в контейнере от непривилегированного пользователя.
- Резервные копии содержат персональные данные и хэши паролей — храните их как базу.
- Сессии хранятся в базе в виде хэшей; отключение сотрудника завершает его сессии сразу.

12 Масштабирование

- Процессы backend: `GUNICORN_WORKERS` (по умолчанию по числу ядер). Правки одного плана выполняются по очереди, разных планов — параллельно.
- PostgreSQL 16 в контейнере достаточно для 1000–2000 сотрудников; при выносе на отдельный сервер замените `DATABASE_URL` в окружении backend.
- Ограничение попыток входа считается в памяти каждого процесса backend: фактический предел — до N × `LOGIN_RATE_LIMIT` на учётную запись. Общий строгий предел потребует Redis — он не входит в поставку.

13 Рядом с другими системами на одной машине

«Пульс Проекта» и «По Делу» (и другие системы на Docker) работают на одной виртуальной машине без общих настроек: у каждой свой проект Compose (`puls` у «Пульса»), свои сети, тома и база, свой порт (у «Пульса» 39663, у «По Делу» 39665) и своё имя cookie сессии. Внутренние имена `backend`, `db` и порт 5000 у систем совпадают, но не пересекаются: они видны только внутри сети своей системы.

Проверьте при установке:

1. **Диапазон адресов сети Docker.** При первом запуске Docker сам выбирает для сети `puls_default` диапазон из 172.17–172.31.x.x (65 536 адресов). Если в этот диапазон попадает часть сети компании — DNS-сервер, обратный прокси перед доменом или сеть, откуда заходят сотрудники, — сервер начинает отправлять ответы этим адресам в сеть Docker: перестают работать доменные имена и доступ к другим системам на машине, хотя сами контейнеры исправны. Признак — всё восстанавливается после `docker compose down`. Решение — задать «Пульсу» небольшой диапазон, которого нет в сети компании (уточните у сетевых администраторов), файлом `docker-compose.override.yml` рядом с `docker-compose.yml` (его не трогает `git pull`):

   ```
   networks:
     default:
       ipam:
         config:
           - subnet: 10.239.63.0/24   # свободный в сети компании диапазон
   ```

   Затем `docker compose down` и `docker compose up -d` (данные сохраняются) и проверка: `docker network inspect puls_default -f '{{range .IPAM.Config}}{{.Subnet}}{{end}}'`. Чтобы так не случилось и с другими системами, администратор Docker может один раз ограничить автоматические диапазоны свободным блоком — `"default-address-pools"` в `/etc/docker/daemon.json` (нужен перезапуск Docker, перезапустятся все контейнеры машины).

2. **Порты.** `HTTP_PORT` у систем разные; HTTPS-прокси на сервере (nginx, Caddy) направляет каждый домен на свой порт. Порт 80 и 443 «Пульс» не занимает.

3. **Процессы backend.** По умолчанию `GUNICORN_WORKERS` — 2 × число ядер + 1 (не больше 9). Если машина общая, задайте число явно, например `GUNICORN_WORKERS=4`, чтобы системы не делили память впритык.

4. **Связь «Пульса» и «По Делу»** (ссылки из задач, загрузка структуры и сотрудников). Backend каждой системы работает в своём контейнере и обращается к другой сам, поэтому адреса, которые открывают сотрудники, ему часто не подходят:

   - `localhost` и `127.0.0.1` внутри контейнера — сам контейнер, а не машина: «Адрес «По Делу»» вида `http://localhost:39665` в браузере открывается, а проверка связи из «Пульса» получает отказ в подключении;

- имена, прописанные только в `/etc/hosts` машины, контейнеры не видят — проверка пишет, что имя не находится;
- внутренние имена (`backend`) другой системы не видны: сети Docker у систем разные.

Поэтому в 0.5.1 у подключений есть необязательное поле **Адрес для запросов сервера**: запросы сервера идут на него, а ссылки для сотрудников — по-прежнему на основной адрес. Для двух систем на одной машине:

ГДЕ	ОСНОВНОЙ АДРЕС (ДЛЯ СОТРУДНИКОВ)	АДРЕС ДЛЯ ЗАПРОСОВ СЕРВЕРА
«Пульс»: Интеграции → «По Делу» (оба блока)	`https://podelu.company.ru` или `http://<IP машины>:39665`	`http://host.docker.internal:39665`
«По Делу»: Настройки → Интеграции → «Пульс проекта»	`https://puls.company.ru` или `http://<IP машины>:39663`	`http://host.docker.internal:39663`

Имя `host.docker.internal` ведёт из контейнера на саму машину; в `docker-compose.yml` обеих систем (начиная с «Пульса» 0.5.1 и «По Делу» 1.4.2) оно задано строкой `extra_hosts: host.docker.internal:host-gateway`. Вместо него подойдёт IP машины в сети компании (`http://10.0.0.5:39665`). Условия:

- порт другой системы опубликован не только на `127.0.0.1`: по умолчанию `HTTP_BIND=0.0.0.0` у «Пульса» и `FRONTEND_BIND_ADDRESS=0.0.0.0` у «По Делу». Порт, закрытый на `127.0.0.1`, из контейнеров не виден — ни по `host.docker.internal`, ни по IP машины. Тогда серверу нужен адрес HTTPS-прокси на этой машине (обычно это и есть «Адрес «По Делу»», отдельный адрес для запросов сервера не нужен); если его имя есть только в `/etc/hosts`, направьте имя на машину в `docker-compose.override.yml` — у сервиса `backend` строка `extra_hosts: ["podelu.company.ru:host-gateway"]`, затем `docker compose up -d`. Прокси должен принимать подключения не только на `127.0.0.1`;
- межсетевой экран машины (ufw, firewalld) пропускает подключения из сетей Docker к этому порту, например `sudo ufw allow from 172.16.0.0/12 to any port 39665 proto tcp` (и 39663 — для «По Делу»);
- если backend выходит в сеть через прокси (`HTTPS_PROXY`), адрес для запросов сервера — в `NO_PROXY` (`NO_PROXY=host.docker.internal,10.0.0.5`);
- если заданы `PODELU_ALLOWED_HOSTS` (у «Пульса») и `PULS_ALLOWED_HOSTS` (у «По Делу»), в них есть и имя из адреса для запросов сервера.

Проверка из контейнера «Пульса» — должен вернуться код 200:

```
cd /opt/puls
sudo docker compose exec backend curl -sS -o /dev/null -w '%{http_code}\n'
http://host.docker.internal:39665/api/health
```

и из контейнера «По Делу» (в её каталоге установки) — `curl … http://host.docker.internal:39663/api/health/ready`. После смены адреса для запросов сервера пароль и ключи нужно ввести заново: они уходят только на адрес, для которого введены. Затем — «Проверить связь» в обеих системах; результат проверки показывает, куда обращался сервер.

14 Установка в закрытом контуре

Если у сервера нет доступа к Docker Hub, PyPI и npm, соберите образы на машине с доступом:

```
cd puls_proekta
docker compose build
docker pull postgres:16-alpine
docker save puls-backend puls-frontend postgres:16-alpine | gzip > puls-0.7.3-images.tar.gz
```

На сервере:

```
gunzip -c puls-0.7.3-images.tar.gz | sudo docker load
cd /opt/puls
sudo docker compose up -d          # без --build: используются загруженные образы
```

Если в компании есть зеркало реестра образов, укажите его в `/etc/docker/daemon.json` (`"registry-mirrors"`).

15 Диагностика

ПРИЗНАК	ПРИЧИНА И РЕШЕНИЕ
`docker compose up` пишет «Задайте SECRET_KEY...»	Не заполнен `.env`
backend перезапускается, в журнале `SECRET_KEY`	Секрет короче 32 символов или значение из примера
backend `unhealthy`, в журнале ошибка миграции	Восстановите копию `pre-update`, верните прежнюю версию, передайте журнал разработчику
Страница открывается, вход «Ошибка сети»	backend не готов: `docker compose ps`, `docker compose logs backend`
После входа снова открывается страница входа	`SESSION_COOKIE_SECURE=true`, а система открыта по HTTP: откройте по HTTPS или верните `false`
Сотрудники домена не входят	«Интеграции → Вход через домен → Проверить подключение»; доступность сервера каталога из контейнера
backend перезапускается, в журнале `BOOTSTRAP_ADMIN_PASSWORD: Пароль должен быть не короче 8 символов`	База пустая, а пароль первого администратора короткий. Новая установка — задайте пароль длиннее; переход с 1.0.x — оставьте `BOOTSTRAP_ADMIN_PASSWORD` пустым и перенесите базу (`docker compose --profile migrate run --rm legacy-import`), затем `docker compose up -d`
После запуска «Пульса» перестали работать домены или другие системы на машине; после `docker compose down` всё вернулось	Диапазон адресов сети Docker совпал с сетью компании — задайте свой диапазон (раздел «Рядом с другими системами на одной машине»)
Перенос 1.0.x: «В базе 0.2 уже есть данные»	Перенос — только в пустую базу. Для повторной репетиции используйте `--dry-run`. Если в базе только администратор из `BOOTSTRAP_ADMIN_PASSWORD` (backend запускали до переноса) — повторите перенос с `--replace`
«По Делу»: проверка связи пишет «localhost: для сервера «Пульса» это он сам», «отклонил подключение» или «не находит имя», хотя в браузере «По Делу» открывается	Сервер «Пульса» в контейнере не видит адрес, который открывают сотрудники. Укажите **Адрес для запросов сервера** `http://host.docker.internal:39665` или IP машины (раздел «Рядом с другими системами на одной машине», пункт 4) и введите ключ или пароль заново

ПРИЗНАК	ПРИЧИНА И РЕШЕНИЕ
«По Делу»: «не ответила вовремя» по `host.docker.internal` или IP машины	Межсетевой экран машины не пропускает подключения из сетей Docker к порту «По Делу», или запрос ушёл в прокси сервера — разрешите порт для 172.16.0.0/12 и добавьте адрес в `NO_PROXY`
«По Делу»: `host.docker.internal` не находится	`docker-compose.yml` старше 0.5.1: обновите «Пульс» или добавьте backend `extra_hosts: ["host.docker.internal:host-gateway"]` и выполните `docker compose up -d`
Jira: «Соединение установлено», но задачи в планах не загружаются	Обновите «Пульс» до 0.3.1 или новее; «Проверить подключение» теперь проверяет и поиск задач и показывает версию Jira
Перенос 1.0.x: код 2, «Расхождения»	Ничего не записано; отчёт `import/import-report.md` — разработчику
Нет места на диске	`docker system df`; старые копии в `BACKUP_DIR`; `docker image prune`
Загрузка из MS Project: «Слишком большой запрос»	Файл больше 25 МБ: увеличьте `MAX_CONTENT_LENGTH` (backend, байты) и `client_max_body_size` в `nginx.conf` (frontend), пересоберите и перезапустите
Загрузка из MS Project: «Это не файл MS Project XML»	Сохраните проект в MS Project как XML: «Файл → Сохранить как → XML (*.xml)»; файл `.mpp` пока не читается

16 Справочник команд

```
# состояние и журналы
sudo docker compose ps
sudo docker compose logs -f backend

# обновление
sudo sh deploy/update.sh <версия>

# резервные копии
sudo docker compose exec backup sh /usr/local/bin/backup.sh now <метка>
sudo docker compose exec backup sh /usr/local/bin/backup.sh list
sudo docker compose exec backup sh /usr/local/bin/backup.sh restore <файл>   # backend
остановлен

# перенос 1.0.x (один раз)
sudo docker compose --profile migrate run --rm legacy-import
sudo docker compose --profile migrate run --rm legacy-import \
  python scripts/import_legacy.py /import/puls.db --dry-run --report /import/rehearsal-
report.md

# обслуживание backend
sudo docker compose exec backend flask --app wsgi reset-password <логин>
sudo docker compose exec backend flask --app wsgi purge-sessions
sudo docker compose exec backend flask --app wsgi calendar-seed
sudo docker compose exec backend alembic current
```